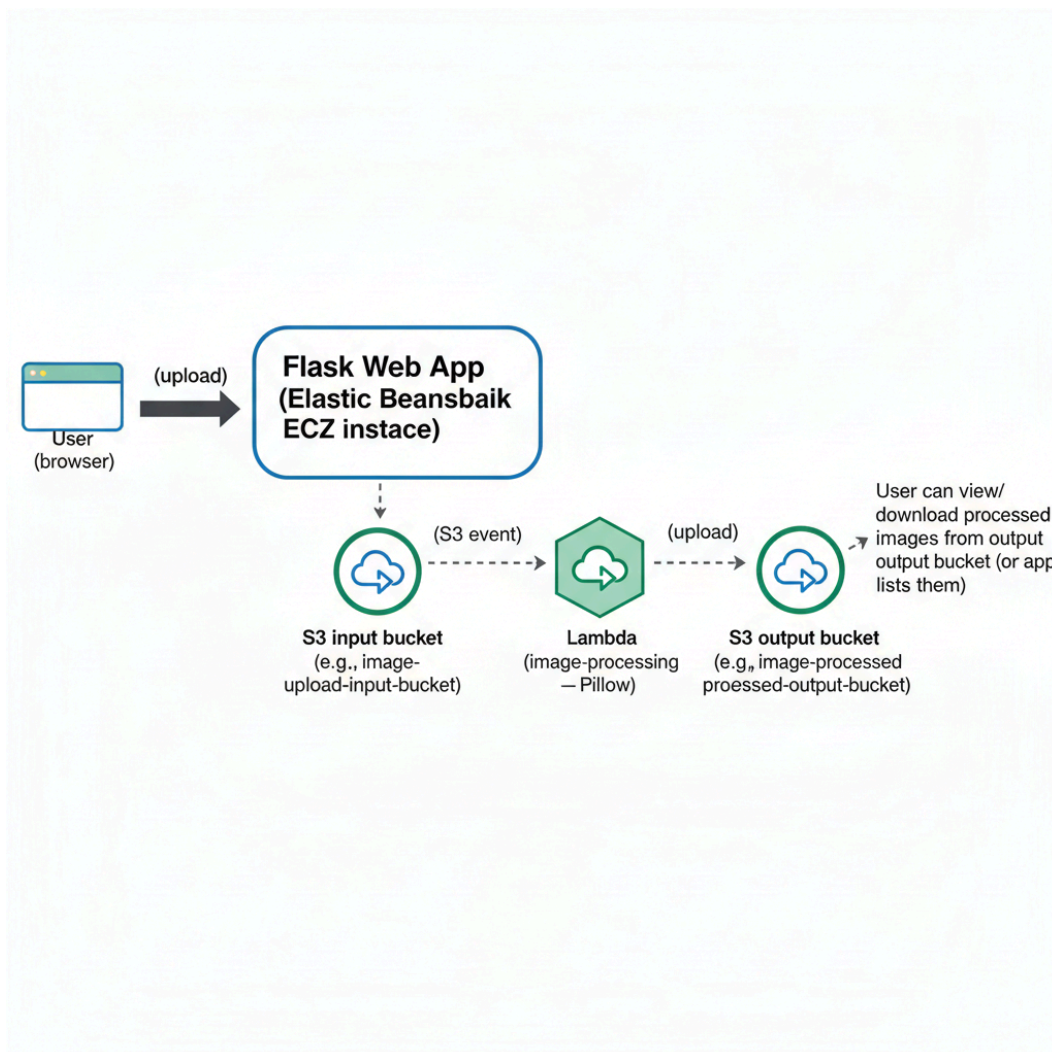


Image Upload & Processing using Elastic Beanstalk + S3 + Lambda

Short summary (one-liner): A Flask web app hosted on **Elastic Beanstalk** uploads images to an **S3 input bucket**. An **AWS Lambda** function is triggered on new uploads, processes the image (resize / optional watermark / convert), and writes the result to an **S3 output bucket**. Logs and monitoring via **CloudWatch**.

1) Architecture



2) Prerequisites

- AWS account with permission to create S3, Lambda, IAM roles, Elastic Beanstalk, CloudWatch.
- Local machine: Python 3.9+ (I use 3.9 in examples), pip, git.
- AWS CLI configured (`aws configure`) with your credentials and region (e.g., `ap-south-1`).
- (Optional but recommended) Elastic Beanstalk CLI: `pip install awsebcli --upgrade --user` and add `~/.local/bin` to PATH (or install via package manager).
- Basic knowledge of Flask, boto3, and AWS console/CLI.

3) Resource names & variables (change these to your names)

Use consistent names (no spaces):

- `REGION = ap-south-1` (example)
- `INPUT_BUCKET = image-upload-input-bucket-<your-unique-suffix>`
- `OUTPUT_BUCKET = image-processed-output-bucket-<your-unique-suffix>`
- `LAMBDA_ROLE = lambda-s3-exec-role`
- `EB_APP_NAME = image-processing-app`
- `EB_ENV_NAME = image-processing-env`
- `EB_INSTANCE_PROFILE = eb-s3-instance-profile`

Important: S3 bucket names must be globally unique. Add a random suffix or your initials.

4) Step A — Create S3 buckets

Via AWS Console (quick)

Go to S3 → Create bucket. Create one bucket for input and one for output. Example:

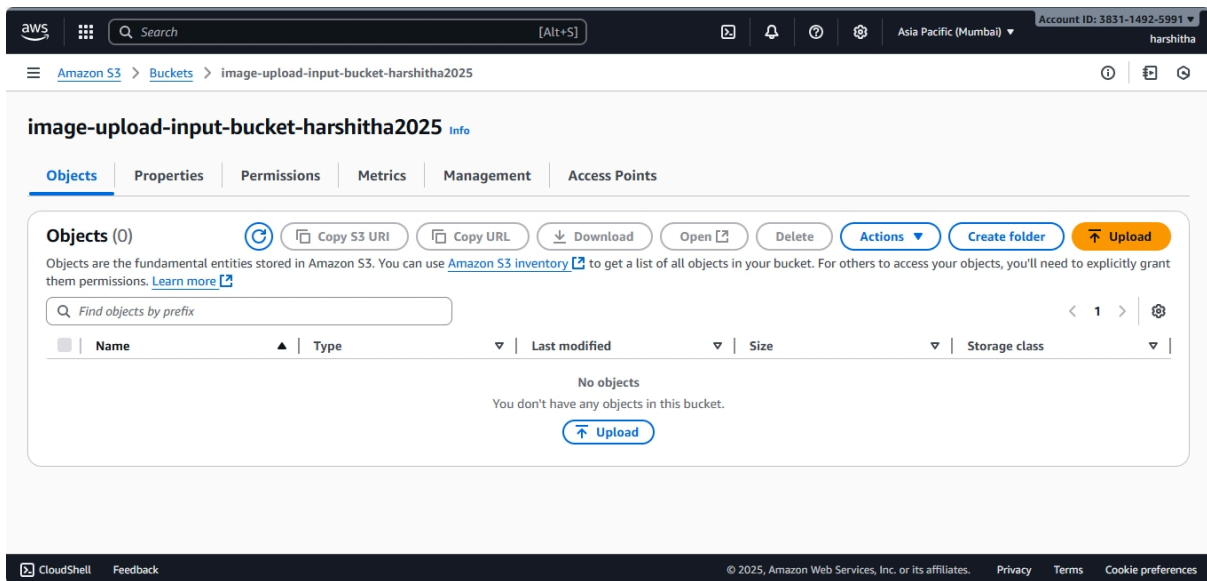
- image-upload-input-bucket-harshitha-2025
- image-processed-output-bucket-harshitha-2025

Create First Bucket (Input Bucket)

1. Click **Create bucket** button.
2. **Bucket name:**
 - Give a unique name (e.g., image-upload-input-bucket-harshitha2025)
 - S3 bucket names must be globally unique.
3. **Region:**
 - Select your preferred AWS region (example: Asia Pacific (Mumbai) ap-south-1).
4. **Bucket settings for Block Public Access:**
 - Keep all checked (recommended for security).
 - You can customize later if needed.
5. Click **Create bucket**.

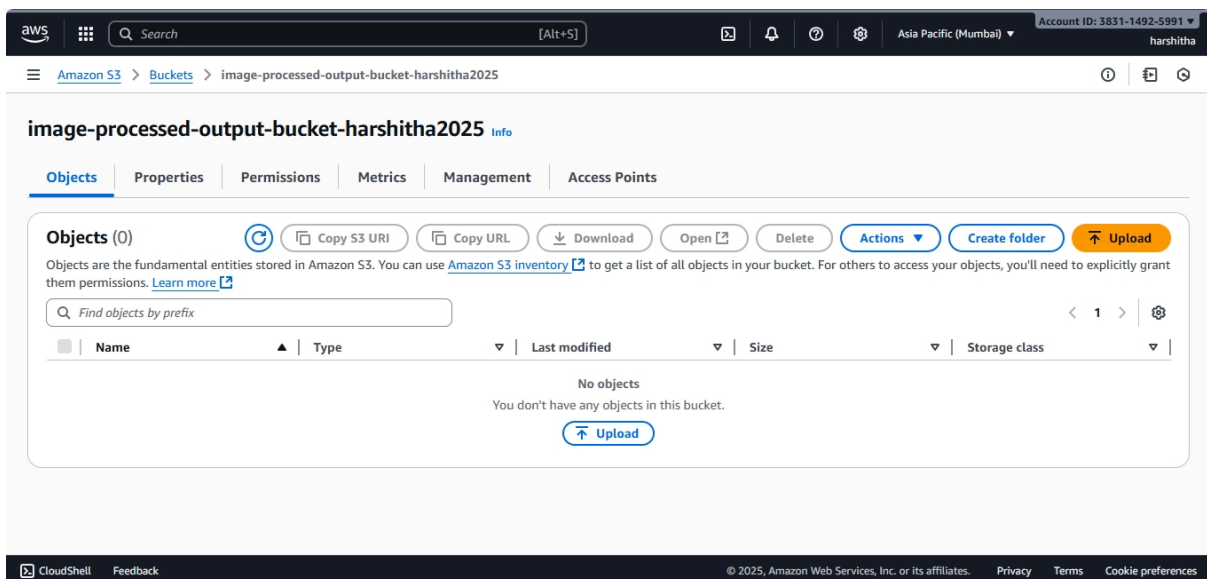
Via AWS CLI (example):

```
aws s3api create-bucket --bucket ${INPUT_BUCKET} --region ${REGION}
--create-bucket-configuration LocationConstraint=${REGION}
aws s3api create-bucket --bucket ${OUTPUT_BUCKET} --region ${REGION}
--create-bucket-configuration LocationConstraint=${REGION}
```



Create Second Bucket (Output Bucket)

1. Click **Create bucket** again.
2. **Bucket name:**
 - Example: image-processed-output-bucket-harshitha2025.
3. Select the same region as input bucket.
4. Keep **Block Public Access** settings as is.
5. Click **Create bucket**.



5) Step B — IAM roles & policies

Step B.1 — Create Lambda Execution Role (via Console)

1. Open IAM Console

- Login to AWS Console → search **IAM** → click **IAM**.

2. Create Role

1. Click **Roles** in the left menu → click **Create role**.
2. Select **AWS Service** → choose **Lambda** → click **Next**.

3. Attach Permissions Policy

- Click **Create policy** (this will allow us to create a custom policy).
- Choose **JSON tab** → paste your S3 + CloudWatch inline policy (update variables):

```
{  
  
  "Version": "2012-10-17",  
  
  "Statement": [  
  
    {  
  
      "Effect": "Allow",  
  
      "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],  
  
      "Resource": [  
  
        "arn:aws:s3:::YOUR_INPUT_BUCKET_NAME",  
  
        "arn:aws:s3:::YOUR_INPUT_BUCKET_NAME/*",  
  
        "arn:aws:s3:::YOUR_OUTPUT_BUCKET_NAME",  
  
        "arn:aws:s3:::YOUR_OUTPUT_BUCKET_NAME/*"  
  
      ]  
  
    }  
  
  ]  
}
```

```

    },
    {
      "Effect": "Allow",
      "Action":
        ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:*:*:*"
    }
  ]
}

```

- Click **Review policy**, give it a name like LambdaS3CloudWatchPolicy, and **Create policy**.

4. Attach the Policy to Role

- Back to role creation → Search for your newly created policy → select it.
- Click **Next: Tags** (optional) → **Next: Review**.
- Role name: LambdaExecutionRole → click **Create role**.

The screenshot shows the AWS IAM console 'Create role' wizard, specifically Step 3: Name, review, and create. The interface is divided into several sections:

- Role details:** Includes a 'Role name' field with the value 'LambdaExecutionRole' and a 'Description' field with the text 'Allows Lambda functions to call AWS services on your behalf.'
- Step 1: Select trusted entities:** Shows a 'Trust policy' section with a JSON snippet:


```

1 = {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": "lambda.amazonaws.com"
8       }
9     }
10  ]
11 }
12
13
14
15
16 ]

```
- Step 2: Add permissions:** Shows a 'Permissions policy summary' table with one entry:

Policy name	Type	Attached as
LambdaS3CloudWatchPolicy	Customer managed	Permissions policy
- Step 3: Add tags:** Includes a section for 'Add tags - optional' with a note that tags are key-value pairs and a button to 'Add new tag'.

At the bottom right, there are three buttons: 'Cancel', 'Previous', and 'Create role'.

Step B.2 — Create Elastic Beanstalk Instance Role

1. Go to IAM Console → Roles → Create Role

1. Click **Create role**.
2. Select **AWS Service** → choose **EC2** → click **Next**.

2. Create Custom Policy

- Again click **Create policy** → JSON tab → paste:

```
{  
  
  "Version": "2012-10-17",  
  
  "Statement": [  
  
    {  
  
      "Effect": "Allow",  
  
      "Action": ["s3:PutObject", "s3:PutObjectAcl", "s3:GetObject"],  
  
      "Resource": ["arn:aws:s3:::YOUR_INPUT_BUCKET_NAME/*"]  
  
    }  
  
  ]  
  
}
```

- Name it: EBS3Policy → **Create policy**.

3. Attach Policy to Role

- Back to role creation → Search for EBS3Policy → select it → **Next: Tags** → **Next: Review**.
- Role name: EBInstanceRole → click **Create role**.

Step 1

Select trusted entity

Step 2

Add permissions

Step 3

Name, review, and create

Name, review, and create

Role details

Role name

Enter a meaningful name to identify this role.

EBSInstanceRole

Maximum 64 characters. Use alphanumeric and "+=,@-_" characters.

Description

Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=.@-~\![]{}#%&*()';:~'

Step 1: Select trusted entities

Edit

Trust policy

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": [  
7         "ec2:DescribeInstances"8     ]  
9   }  
10 ]  
11 }
```

Step 2: Add permissions

Edit

Permissions policy summary

Policy name	Type	Attached as
EBS3Policy	Customer managed	Permissions policy

Step 3: Add tags

Add tags - optional

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Step B.3 – Create Elastic Beanstalk Environment

1.Access Elastic Beanstalk Console

- Navigate to the AWS Management Console.
- In the search bar, type **Elastic Beanstalk** and select it.

2. Create a New Application

1. **Click on Create Application.**
2. **Application Name:** Enter a name for your application (e.g., ImageProcessingApp).

3. **Environment Tier:** Choose **Web server environment**.
4. **Platform:** Select **Python** (e.g., Python 3.9 or the latest available).
5. **Application Code:** Choose **Sample application** for now; we'll deploy our own later.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Account ID: 3831-1492-5991

harshitha

Elastic Beanstalk > Create environment

Step 1

Configure environment

Step 2

Configure service access

Step 3 - optional

Set up networking, database, and tags

Step 4 - optional

Configure instance traffic and scaling

Step 5 - optional

Configure updates, monitoring, and logging

Step 6

Review

Configure environment

Info

Environment tier

Info

Amazon Elastic Beanstalk has two types of environment tiers to support different types of web applications.

☒ Web server environment

Run a website, web application, or web API that serves HTTP requests. [Learn more](#)

☐ Worker environment

Run a worker application that processes long-running workloads on demand or performs tasks on a schedule. [Learn more](#)

Application information

Info

Application name

ImageProcessingApp

Maximum length of 100 characters.

Application tags (optional)

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Choose the name, subdomain and description for your environment. These cannot be changed later.

Environment name

ImageProcessingApp-env

Must be from 4 to 40 characters in length. The name can contain only letters, numbers, and hyphens. It can't start or end with a hyphen. This name must be unique within a region in your account.

Domain

Leave blank for autogenerated value

.ap-south-1.elasticbeanstalk.com

Check availability

Environment description

Platform

Info

Platform

Python

Platform branch

Python 3.13 running on 64bit Amazon Linux 2023

Platform version

4.7.2 (Recommended)

Application code

Info

☒ Sample application

☐ Existing version

Application versions that you have uploaded.

☐ Upload your code

Upload a source bundle from your computer or copy one from Amazon S3.

Presets

Info

Start from a preset that matches your use case or choose custom configuration to unset recommended values and use the service's default values.

Configuration presets

☒ Single instance (free tier eligible)

☐ Single instance (using spot instance)

☐ High availability

☐ High availability (using spot and on-demand instances)

☐ Custom configuration

Cancel

Next

3. Configure service access

3.1: Service Role

- **Service Role:** Select `aws-elasticbeanstalk-service-role` (default).

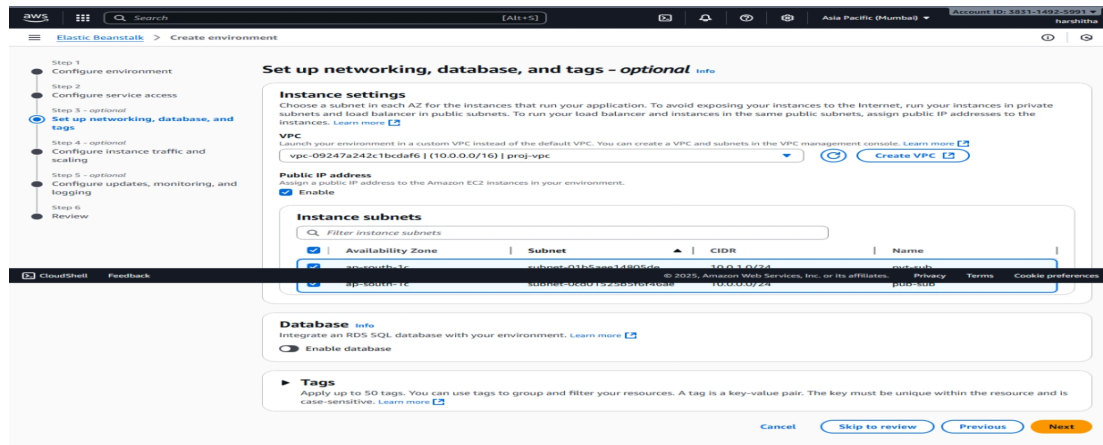
3.2: Instance Profile

- **Instance Profile:** Select `EBInstanceProfile` (the IAM role you created earlier).

The screenshot shows the AWS Management Console interface for the 'Create environment' wizard. The left sidebar contains a progress bar with six steps: Step 1: Configure environment, Step 2: Configure service access (highlighted with a blue circle), Step 3 - optional: Set up networking, database, and tags, Step 4 - optional: Configure instance traffic and scaling, Step 5 - optional: Configure updates, monitoring, and logging, and Step 6: Review. The main content area is titled 'Configure service access' with an 'Info' link. It contains three sections: 'Service access' with a description of IAM roles and a 'Learn more' link; 'Service role' with a dropdown menu showing 'aws-elasticbeanstalk-service-role' and a 'Create role' button; 'EC2 instance profile' with a dropdown menu showing 'EBInstanceRole' and a 'Create role' button; and 'EC2 key pair - optional' with a dropdown menu showing 'learning' and a 'Create role' button. At the bottom of the main content area are buttons for 'Cancel', 'Skip to review', 'Previous', and 'Next'. The footer of the console shows 'CloudShell', 'Feedback', and copyright information for Amazon Web Services.

3.3: Network

- **VPC:** Choose the default VPC or a custom VPC if you have one.
- **Subnets:** Select the appropriate subnets for your EC2 instances.



3.4: Security

- **Security Groups:** Choose the default security group or a custom one.
- **IAM Role:** Ensure the EC2 instance profile is set to EBInstanceProfile.

3.5: Load Balancer

- **Load Balancer:** Choose whether to use an Application Load Balancer (ALB) or Network Load Balancer (NLB), depending on your application's requirements.

3.6: Scaling

- **Scaling:** Configure the desired number of instances and scaling policies based on your application's needs.

3.7: Monitoring

- **Monitoring:** Enable enhanced monitoring if required.

4. Configure Updates, Monitoring, and Logging

1. **Managed Updates:** Enable or disable managed platform updates.
2. **Monitoring:** Enable enhanced monitoring if needed.
3. **Logging:** Enable log streaming to CloudWatch for better visibility.

5. Review and Create Environment

1. **Review** all the configurations.
2. **Click on Create environment.**

AWS Elastic Beanstalk will now provision the necessary resources, including EC2 instances, security groups, and load balancers, to run your application. This process may take a few minutes.

Review [Info](#)

Step 1: Configure environment [Edit](#)

Environment information

Environment tier Web server environment	Application name ImageProcessingApp
Environment name ImageProcessingApp-env	Application code Sample application
Platform arn:aws:elasticbeanstalk:ap-south-1:platform/Python 3.13 running on 64bit Amazon Linux 2023/4.7.2	

Step 2: Configure service access [Edit](#)

Service access [Info](#)

Your EC2 instances:

Service role arn:aws:iam::383114925991:role/aws-elasticbeanstalk-service-role	EC2 key pair learning	EC2 instance profile EBInstanceProfile
---	---------------------------------	--

Step 3: Set up networking, database, and tags [Edit](#)

Networking, database, and tags [Info](#)

Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that's integrated with your environment.

Network

VPC vpc-09247a242c1bcdaf6	Public IP address true	Instance subnets subnet-01b5aee14805de7da,subnet-0cd01525b5f6f46ae
-------------------------------------	----------------------------------	--

Tags

Key	Value
No tags There are no tags defined	

Step 4: Configure instance traffic and scaling [Edit](#)

Instance traffic and scaling [Info](#)

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances

IMDSv1
Disabled

Capacity

Environment type Single instance	Fleet composition On-Demand instance	On-demand base 0
On-demand above base 70	Capacity rebalancing Disabled	Scaling cooldown 360
Processor type x86_64	Instance types t3.micro,t3.small	AMI ID ami-05c84b9559413a31e

Step 5: Configure updates, monitoring, and logging [Edit](#)

Updates, monitoring, and logging [Info](#)

Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring

System enhanced	Cloudwatch custom metrics - instance Retention 7	Cloudwatch custom metrics - environment Lifecycle false
---------------------------	---	--

Log streaming
Disabled

Updates

Managed updates Enabled	Deployment batch size 100	Deployment batch size type Percentage
Command timeout 600	Deployment policy AllAtOnce	Health threshold Ok
Ignore health check false	Instance replacement false	

Platform software

Lifecycle false	Log streaming Disabled	NumProcesses 1
NumThreads 15	WSGIPath application	Proxy server nginx
Logs retention 7	Rotate logs Disabled	Update level minor

X-Ray enabled
Disabled

Environment properties

Source	Key	Value
Plain text	PYTHONPATH	/var/app/venv/staging-LQM1test/bin

[Cancel](#) [Previous](#) [Create](#)

6) Step C — Build & deploy Lambda (with Pillow dependency)

Why special handling? Pillow has native binaries; simplest approaches:

(recommended): Create a Lambda Layer that contains Pillow compiled for the Lambda runtime, then attach that layer to your Lambda function. This keeps your function code small.

1) Build `pillow-layer.zip` (recommended: Docker or WSL — Amazon-Linux compatible)

Important: If you build on plain Windows without WSL/Docker, Pillow's native binaries likely won't work in Lambda. Use WSL or Docker.

A) Using Docker (recommended, works Windows/macOS/Linux)

Open terminal (PowerShell / CMD / WSL) in a new empty folder for the layer.

```
# create a folder that will contain python packages

mkdir python

# run Amazon Lambda Python 3.9 image, install Pillow into /asset/python

docker run --rm -v "$PWD":/asset public.ecr.aws/lambda/python:3.9 \

    /bin/sh -c "pip install Pillow -t /asset/python"

# zip the folder (creates pillow-layer.zip)

zip -r pillow-layer.zip python
```

- After success, `pillow-layer.zip` will be in the folder. This zip structure must have top-level `python/` directory.

B) Using WSL (if you have WSL on Windows)

Open WSL terminal in project folder:

```
mkdir python

pip install --target=./python Pillow

zip -r pillow-layer.zip python
```

```
Ubuntu
updating: python/PIL/ImageMorph.py (deflated 69%)
updating: python/PIL/PcxImagePlugin.py (deflated 63%)
updating: python/PIL/McIdasImagePlugin.py (deflated 55%)
updating: python/PIL/ExifTags.py (deflated 66%)
updating: python/PIL/ImageDraw.py (deflated 79%)
updating: python/PIL/SgiImagePlugin.py (deflated 64%)
updating: python/PIL/TiffTags.py (deflated 65%)
updating: python/PIL/_imagingtk.py (stored 0%)
updating: python/PIL/CurImagePlugin.py (deflated 57%)
updating: python/PIL/ImageMode.py (deflated 66%)
updating: python/PIL/GimpPaletteFile.py (deflated 57%)
updating: python/PIL/ImageShow.py (deflated 74%)
updating: python/PIL/GimpGradientFile.py (deflated 64%)
updating: python/PIL/WalImageFile.py (deflated 64%)
updating: python/PIL/PcdImagePlugin.py (deflated 54%)
updating: python/PIL/DcxImagePlugin.py (deflated 54%)
updating: python/PIL/_imaging.cpython-312-x86_64-linux-gnu.so (deflated 68%)
updating: python/PIL/Hdf5StubImagePlugin.py (deflated 58%)
updating: python/PIL/ImageSequence.py (deflated 59%)
updating: python/PIL/_avif.py (stored 0%)
updating: python/PIL/ImageFile.py (deflated 72%)
updating: python/PIL/WmfImagePlugin.py (deflated 65%)
updating: python/pillow-11.3.0.dist-info/ (stored 0%)
updating: python/pillow-11.3.0.dist-info/REQUESTED (stored 0%)
updating: python/pillow-11.3.0.dist-info/licenses/ (stored 0%)
updating: python/pillow-11.3.0.dist-info/licenses/LICENSE (deflated 76%)
updating: python/pillow-11.3.0.dist-info/top_level.txt (stored 0%)
updating: python/pillow-11.3.0.dist-info/zip-safe (stored 0%)
updating: python/pillow-11.3.0.dist-info/RECORD (deflated 58%)
updating: python/pillow-11.3.0.dist-info/METADATA (deflated 75%)
updating: python/pillow-11.3.0.dist-info/INSTALLER (stored 0%)
updating: python/pillow-11.3.0.dist-info/WHEEL (deflated 24%)
harsh@DESKTOP-2BU8GU2:~/python$ ls
PIL pillow-11.3.0.dist-info pillow-layer.zip pillow.libs python
harsh@DESKTOP-2BU8GU2:~/python$
```

2) Create Lambda Layer in AWS Console (UI steps)

1. Login to **AWS Console** → **Services** → **Lambda**.
2. In left menu click **Layers**.
3. Click **Create layer**.
4. Fill:
 - **Name:** pillow-layer
 - **Description:** Pillow (PIL) for Python 3.9 - built for Lambda (optional)
 - **Upload a .zip file:** Choose the pillow-layer.zip you created.
 - **Compatible runtimes:** Check **Python 3.9** (or the runtime you'll use).
5. Click **Create**.
6. Note the Layer name and **version** shown — you will attach this version to your function.

aws Search [Alt+S] Asia Pacific (Mumbai) Account ID: 8831-1492-5991 harshitha

Lambda Layers > Create layer

Create layer

Layer configuration

Name
pillow-layer

Description - optional
Pillow (PIL) for Python - built for Lambda

☒ Upload a .zip file
☐ Upload a file from Amazon S3

Choose file

pillow-layer.zip
7.11 MB

For files larger than 10 MB, consider uploading using Amazon S3.

Compatible architectures - optional Info
Choose the compatible instruction set architectures for your layer.

arm64

Compatible runtimes - optional Info
Choose up to 15 runtimes.

Python 3.13

License - optional Info

Cancel Create

Create a simple web app
Learn how to implement common use cases in AWS Lambda.
In this tutorial you will learn how to:
• Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
• Invoke your function through its function URL
[Learn more](#)
[Start tutorial](#)

3) Create (or open) your Lambda function (UI)

If you already have the function, skip creating and open it. If not:

1. Lambda Console → **Functions** → **Create function**.
2. **Author from scratch**
 - **Function name:** image-processor-lambda
 - **Runtime:** Python 3.13
 - **Permissions:** Choose **Use an existing role** and select your LambdaExecutionRole (must have S3 + CloudWatch permissions).
3. Click **Create function**.

Create function [Info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.
image-processor-lambda
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.13

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☒ arm64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

Change default execution role

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

- ☐ Create a new role with basic Lambda permissions
- ☒ Use an existing role
- ☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.
LambdaExecutionRole
[View the LambdaExecutionRole role](#) on the IAM console.

Additional configurations
Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

[Cancel](#) [Create function](#)

Architecture

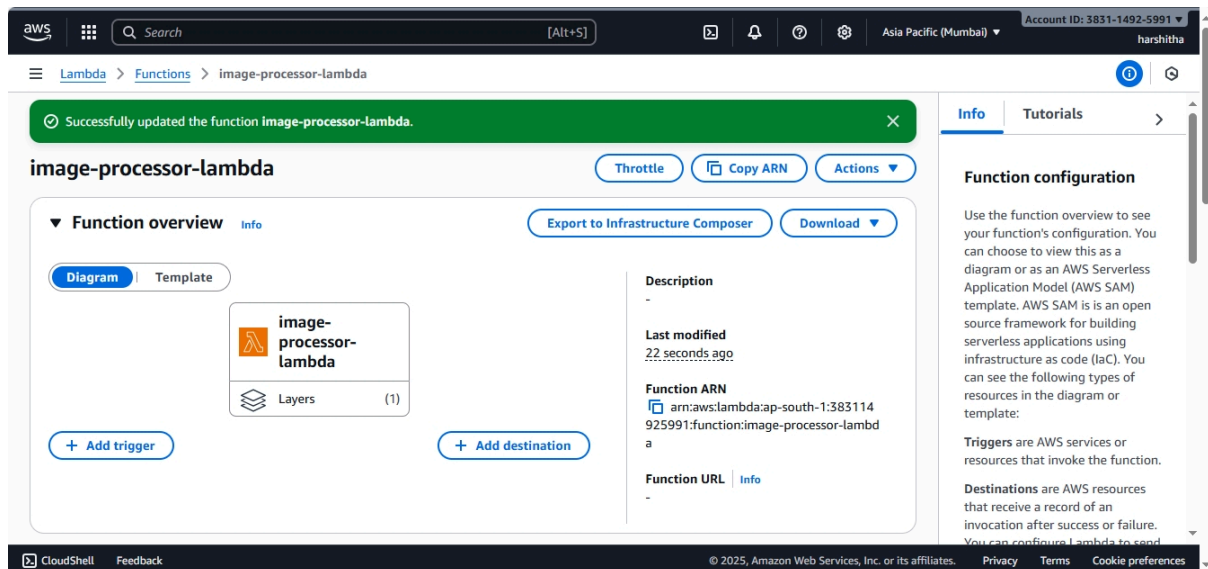
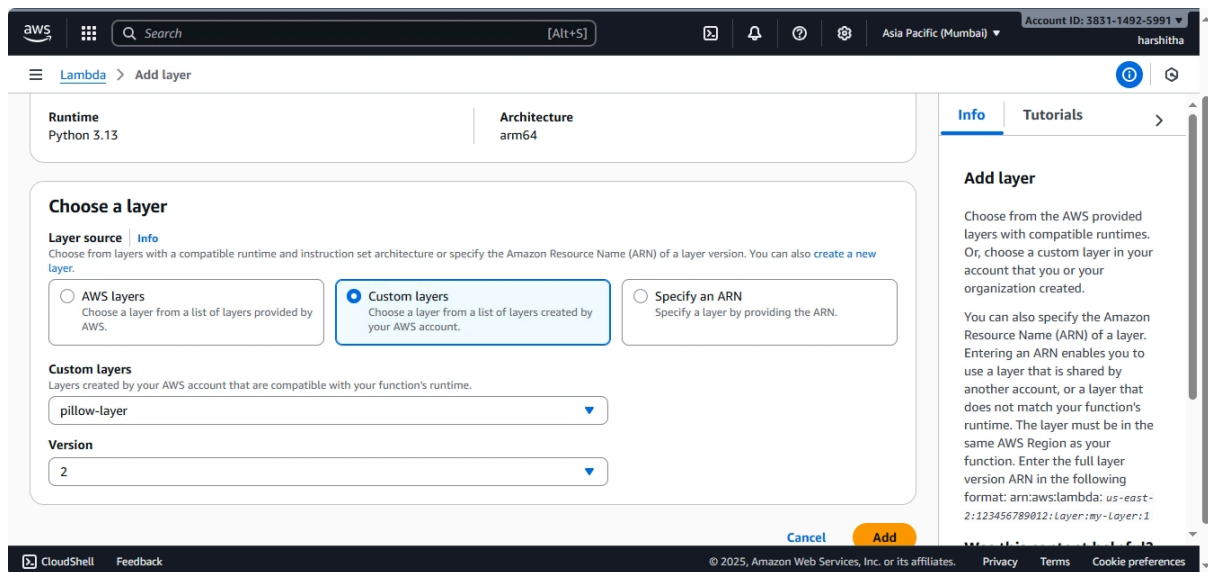
The instruction set architecture of a Lambda function determines the type of computer processor that Lambda uses to run the function. Lambda provides a choice of instruction set architectures:

- arm64 – 64-bit ARM architecture, for the AWS Graviton2 processor.
- x86_64 – 64-bit x86 architecture, for x86-based processors. x86_64 is the default architecture.

Functions that use arm64 architecture offer lower cost per Gb/s compared with the equivalent function running on an

4) Attach the Pillow Layer to your Lambda function (UI)

1. Open the function page → in the **Designer** or **Configuration** area find **Layers**.
2. Click **Add a layer**.
3. Choose **Custom layers** → select pillow-layer and the latest **Version** → **Add**.
4. Save/Confirm.



5) Upload your handler code (UI)

You only need `lambda_function.py` (no Pillow inside — layer provides it).

Option A — paste code in inline editor

1. In function page, under **Code** section click **Edit code** (or Upload).
2. Create/replace `lambda_function.py` with the handler code you already have (you shared it). Example handler name must match `lambda_function.lambda_handler`.

```

import boto3

import os

import urllib.parse

import logging

from PIL import Image, ImageDraw, ImageFont

# Setup logging

logger = logging.getLogger()

logger.setLevel(logging.INFO)

# Initialize S3 client

s3 = boto3.client('s3')

# Destination bucket (set this as environment variable in Lambda config)

OUTPUT_BUCKET = os.environ.get('OUTPUT_BUCKET')

def lambda_handler(event, context):

    try:

        logger.info("Event: %s", event)

# Get source bucket and object key from the S3 event

        record = event['Records'][0]

        src_bucket = record['s3']['bucket']['name']

        src_key = urllib.parse.unquote_plus(record['s3']['object']['key'])

# Local paths inside Lambda /tmp storage

        base_name = os.path.basename(src_key)

        download_path = f'/tmp/{base_name}'

        processed_path = f'/tmp/processed-{base_name}'

# Download the file from source S3

        s3.download_file(src_bucket, src_key, download_path)

        logger.info("Downloaded %s from bucket %s", src_key, src_bucket)

# Open, process, and resize the image

        with Image.open(download_path) as img:

            img = img.convert('RGB') # ensure compatibility

            img.thumbnail((800, 800)) # resize with aspect ratio

# Optional: add watermark

```

```

try:

    draw = ImageDraw.Draw(img)

    font = ImageFont.load_default()

    draw.text((10, 10), 'Processed by EB+Lambda', font=font)

    logger.info("Watermark added successfully")

except Exception as e:

    logger.error("Watermark error: %s", str(e))

# Save processed image

img.save(processed_path, 'JPEG')

# Upload processed file to output bucket under "processed/" folder

dest_key = f'processed/{base_name}'

s3.upload_file(processed_path, OUTPUT_BUCKET, dest_key)

logger.info("Uploaded processed file to %s/%s", OUTPUT_BUCKET, dest_key)

return {

    'statusCode': 200,

    'body': f'Uploaded processed/{base_name} to {OUTPUT_BUCKET}'

}

except Exception as e:

    logger.error("Error processing file: %s", str(e))

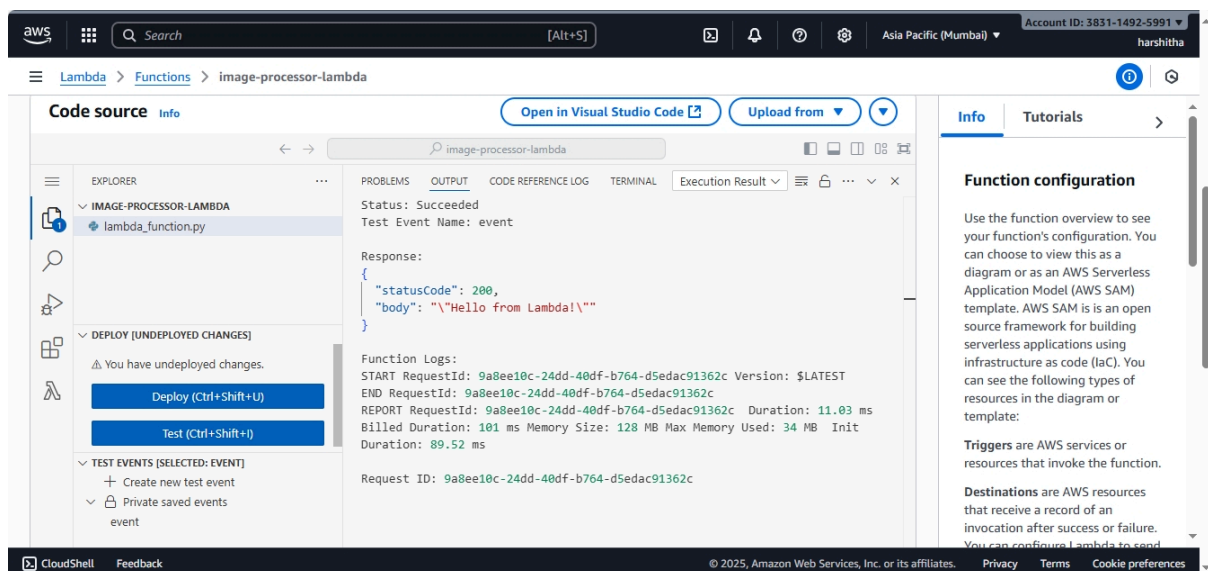
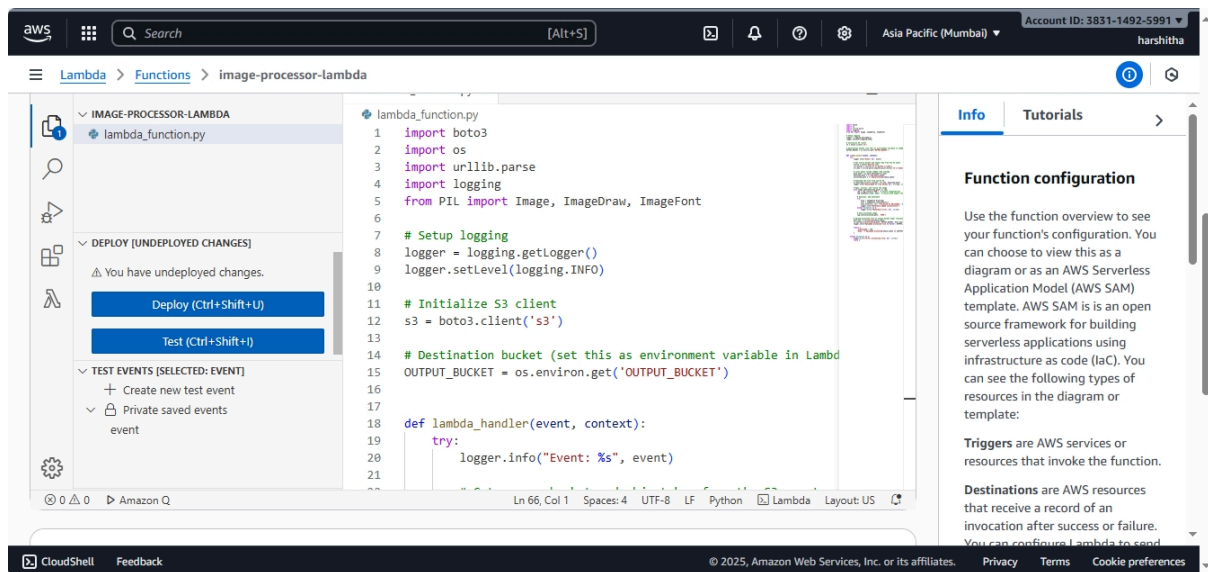
    raise e

```

3. Click **Deploy** (or Save).

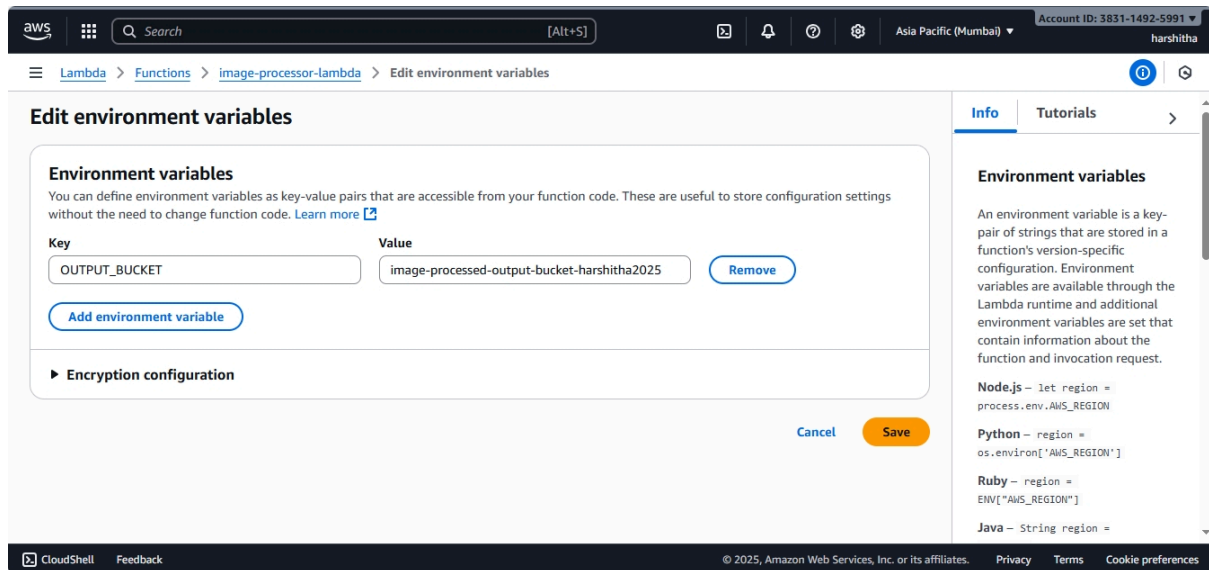
Option B — upload a zip

1. zip function.zip lambda_function.py locally.
2. In Lambda Console → Code → Deploy → Upload from → .zip file → choose function.zip → Deploy.



6) Set environment variable OUTPUT_BUCKET (UI)

1. Function page → **Configuration** → **Environment variables** → **Edit**.
2. Add: OUTPUT_BUCKET = your-output-bucket-name (e.g., image-processed-output-bucket-harshitha2025)
3. Save.



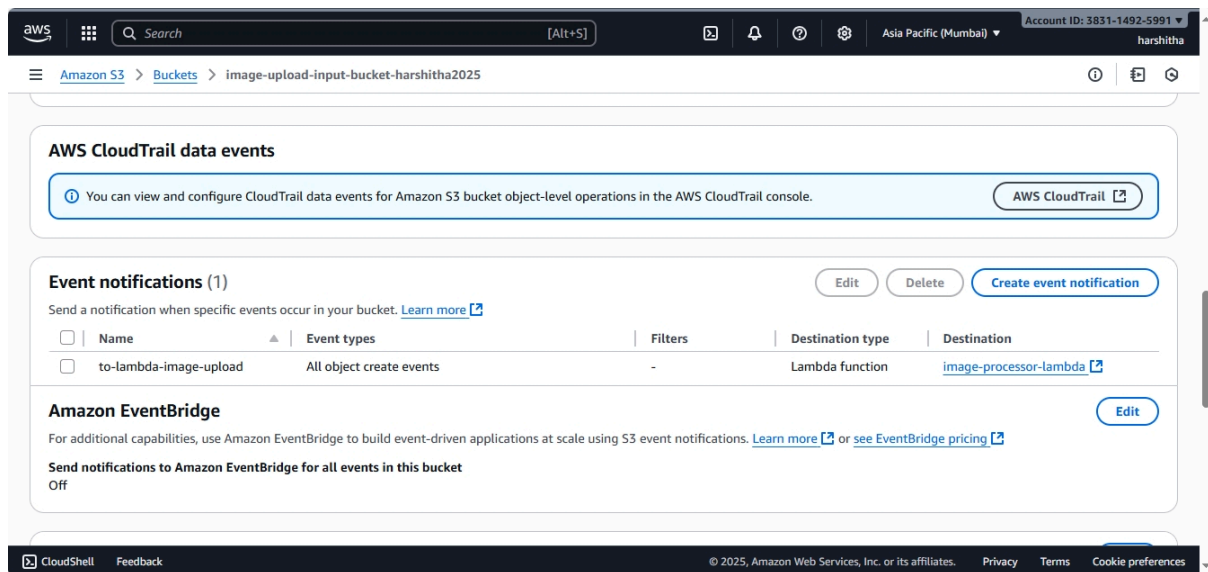
7) Step D — Configure S3 event to trigger Lambda

Console steps (easiest):

1. Open the S3 INPUT_BUCKET → Properties → Event notifications → Create notification.
2. Give it a name (e.g., to-lambda-image-upload).
3. Event type: PUT (or All object create events).
4. Prefix (optional): uploads/ (if you want to only trigger for objects in uploads/).
5. Destination: Lambda function → choose `${LAMBDA_FN_NAME}`. Save.

Note: When you add the notification, S3 will create an invocation permission for Lambda automatically; you may see a statement added to the Lambda resource-based policy.

CLI (example): Use `aws s3api put-bucket-notification-configuration` with the proper JSON — console is easier for first-time users.



8) Step E — Build & deploy Flask app to Elastic Beanstalk

A — Prepare your ZIP (local)

1. Inside your project folder `image-eb-app/` make sure files are at the root of the ZIP (not nested inside another folder). Required files:
 - `application.py`
 - `requirements.txt`
 - `templates/` (folder) with `index.html`
 - optional Procfile (content: `web: gunicorn application:application`)

```

Student_Management_System encoded_key.txt jenkins-key.pem python terraform write_your_first_terraform_project
harsh@DESKTOP-2BUBGU2:~$ cd image-eb-app/
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ ls
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ nano application.py
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ nano requirements.txt
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ mkdir templates/
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ ls
application.py requirements.txt templates
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ cd templates/
harsh@DESKTOP-2BUBGU2:~/image-eb-app/templates$ ls
harsh@DESKTOP-2BUBGU2:~/image-eb-app/templates$ nano index.html
harsh@DESKTOP-2BUBGU2:~/image-eb-app/templates$ cd ..
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ nano Procfile
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ ls
Procfile application.py requirements.txt templates
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ tree
.
├── Procfile
├── application.py
├── requirements.txt
└── templates
    └── index.html

2 directories, 4 files
harsh@DESKTOP-2BUBGU2:~/image-eb-app$ cd ..
harsh@DESKTOP-2BUBGU2:~$ tree image-eb-app/
image-eb-app/
├── Procfile
├── application.py
├── requirements.txt
└── templates
    └── index.html

2 directories, 4 files
harsh@DESKTOP-2BUBGU2:~$

```

2. From the folder that contains `image-eb-app`, create the zip so the files are at top-level:

On macOS / Linux / WSL:

```
cd image-eb-app
zip -r ../image-eb-app.zip *
```

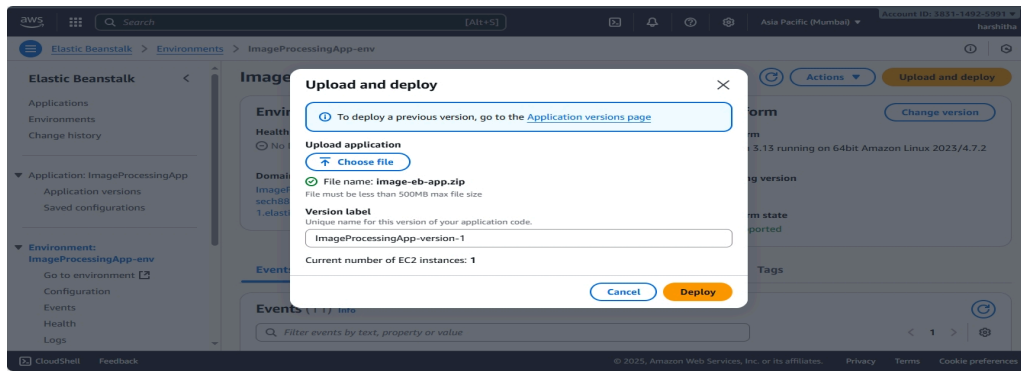
On Windows (PowerShell):

```
cd image-eb-app
Compress-Archive -Path * -DestinationPath ../image-eb-app.zip
```

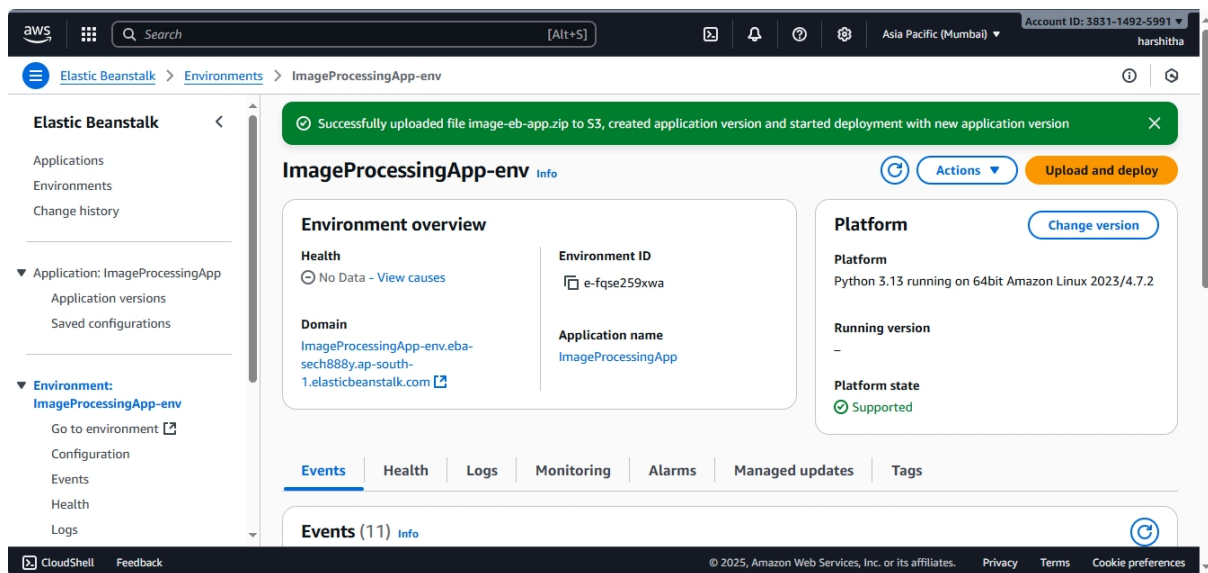
3. Verify: open the zip and confirm you see `application.py`, `requirements.txt`, and `templates/` at the root.

B — Upload & deploy via Elastic Beanstalk Console (UI)

1. Open **AWS Console** → **Elastic Beanstalk**.
2. Click your **Application** `ImageProcessingApp`.
3. Click the **Environment** `ImageProcessingApp-env` (so you are on the environment dashboard).
4. On the environment page look for **Upload and deploy** (button) or **Actions** → **Upload and deploy**.
 - If you see **Upload and deploy** directly: click it.
 - If not, open left menu **Application versions** → click **Upload** then later deploy that version to the environment.
5. In the **Upload and deploy** dialog:
 - **Version label**: enter a meaningful name like `v1-webapp-2025-09-30`.
 - **Source**: choose **Upload** and pick `image-eb-app.zip`.
 - Optionally add a description.



6. Click **Deploy** or **Upload** then **Deploy** (UI may be two-step).
7. Wait — Elastic Beanstalk will upload the zip and deploy. This usually takes **2–6 minutes**. You'll see deployment progress and environment health.



8.1 Project files (structure)

```
image-eb-app/
├── application.py    # flask app (WSGI callable 'application')
├── requirements.txt
├── templates/
│   └── index.html
└── Procfile         # optional, e.g., web: gunicorn application:application
```

8.2 Flask app code — application.py

```
import os
from flask import Flask, render_template, request, redirect, url_for, flash
import boto3
```



```

from werkzeug.utils import secure_filename

application = Flask(__name__)
application.secret_key = os.environ.get('FLASK_SECRET', 'very-secret')

S3_BUCKET = os.environ.get('BUCKET_NAME') # set via eb setenv or EB console
s3 = boto3.client('s3')

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/upload', methods=['POST'])
def upload():
    if 'file' not in request.files:
        flash('No file part')
        return redirect(url_for('index'))
    file = request.files['file']
    if file.filename == '':
        flash('No selected file')
        return redirect(url_for('index'))

    filename = secure_filename(file.filename)
    key = f'uploads/{filename}'
    try:
        # Upload directly using instance role (no keys stored)
        s3.upload_fileobj(file, S3_BUCKET, key)
        flash('Upload successful — processing started!')
    except Exception as e:
        flash('Upload failed: ' + str(e))
    return redirect(url_for('index'))

if __name__ == '__main__':
    application.run(debug=True, host='0.0.0.0', port=5000)

```

8.3 templates/index.html

```

<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <title>Image Upload</title>
</head>
<body>
    <h1>Upload image</h1>
    {% with messages = get_flashed_messages() %}
    {% if messages %}

```

```

<ul>
  {% for m in messages %}
    <li>{{ m }}</li>
  {% endfor %}
</ul>
{% endif %}
{% endwith %}

<form action="{{ url_for('upload') }}" method="post" enctype="multipart/form-data">
  <input type="file" name="file" accept="image/*" required>
  <button type="submit">Upload</button>
</form>
</body>
</html>

```

8.4 requirements.txt

```

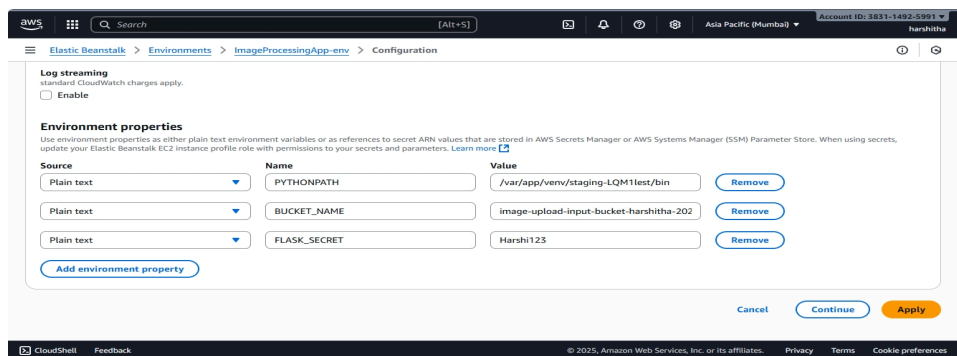
Flask
boto3
unicorn

```

(You don't need Pillow in EB app because processing is done by Lambda.)

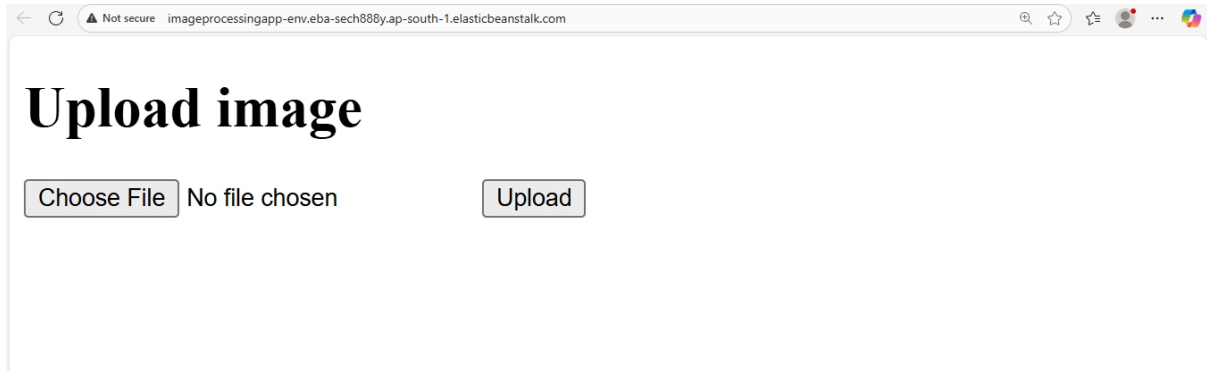
Set environment variables (BUCKET_NAME, FLASK_SECRET)

1. Environment page → **Configuration** → find **Software** card → click **Edit**.
2. Under **Environment properties / Environment variables**, add:
 - BUCKET_NAME = image-upload-input-bucket-<your-suffix>
 - FLASK_SECRET = some-secret-value
3. Click **Apply**. Elastic Beanstalk will redeploy with the new env vars.

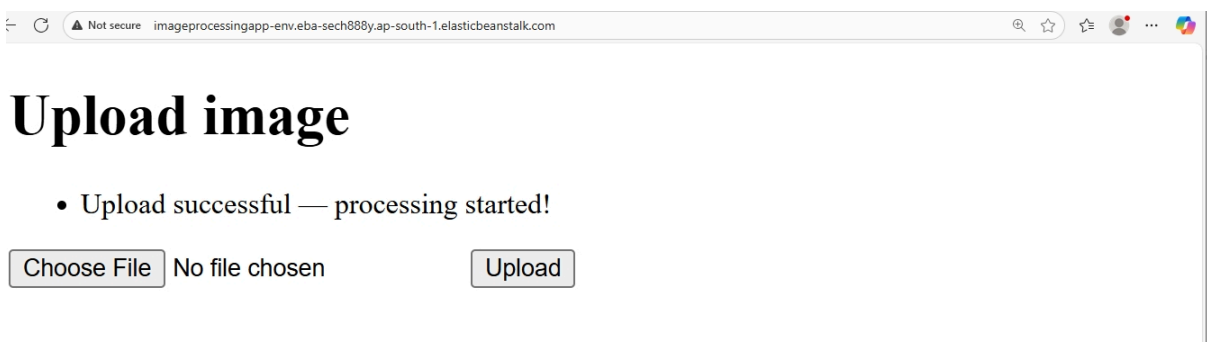
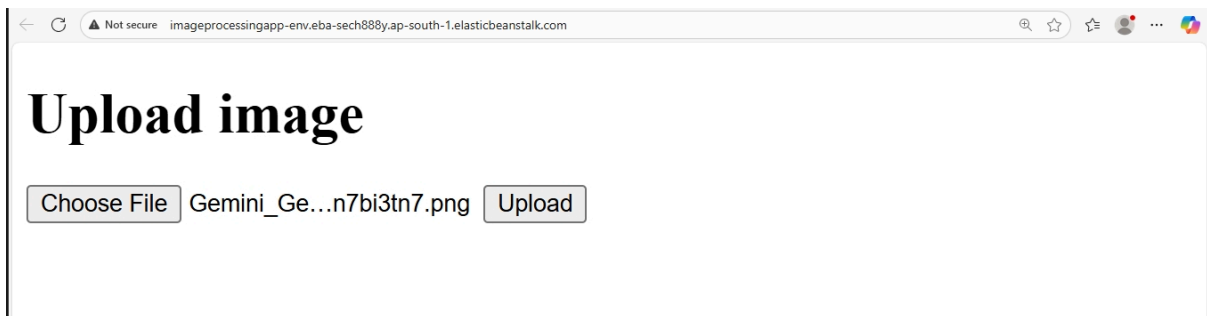


9) Step F — Test the full flow

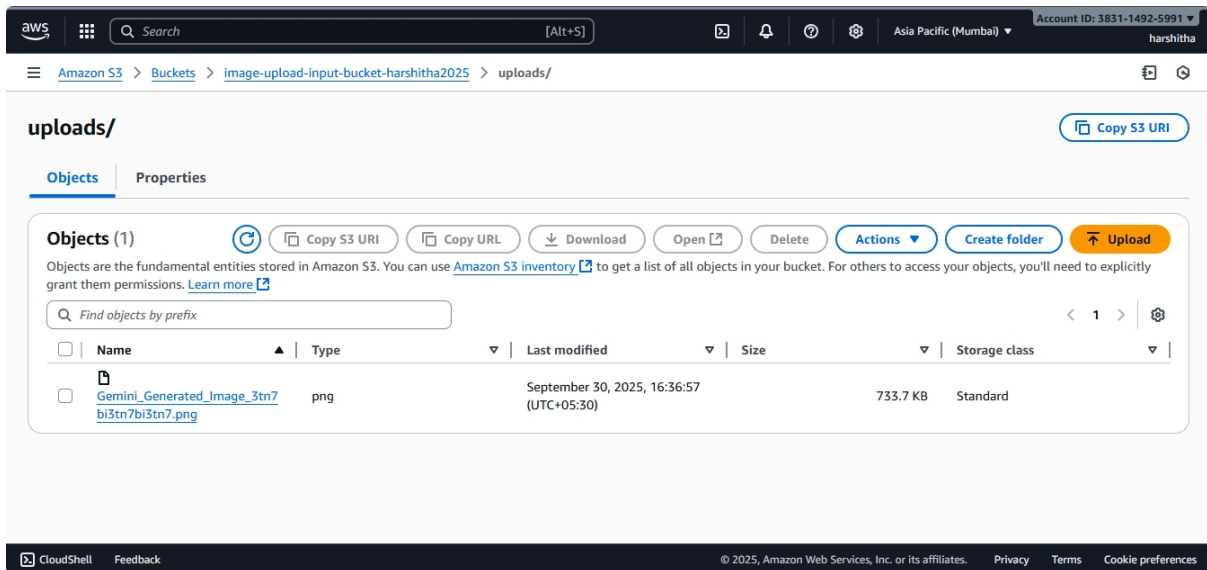
1. Open EB app URL (provided in console or `eb status`).



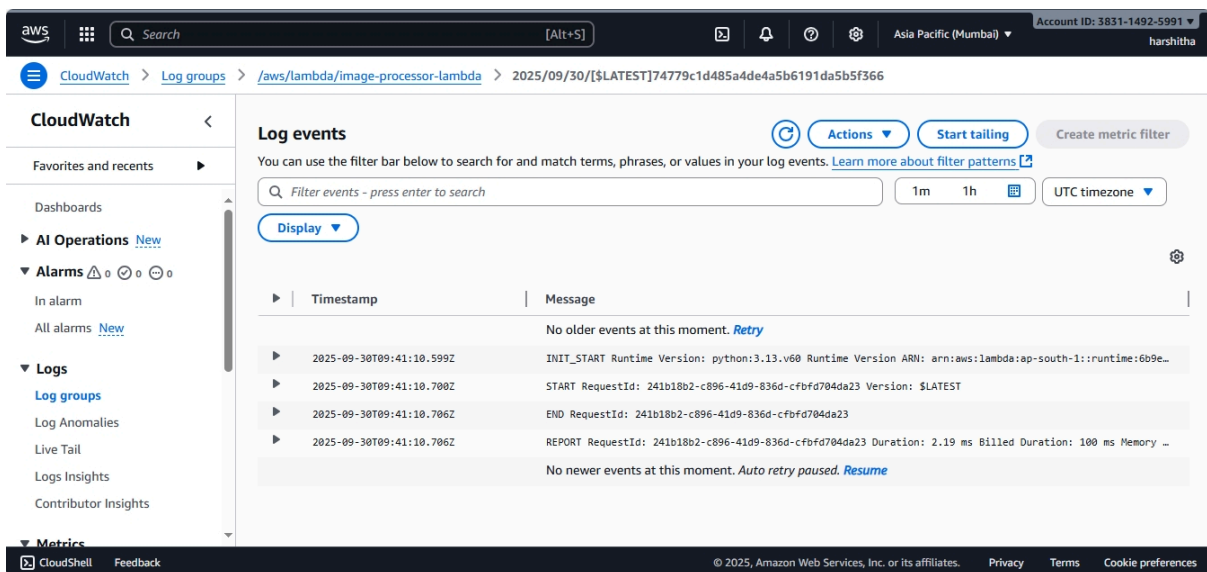
2. Upload an image using the web UI.



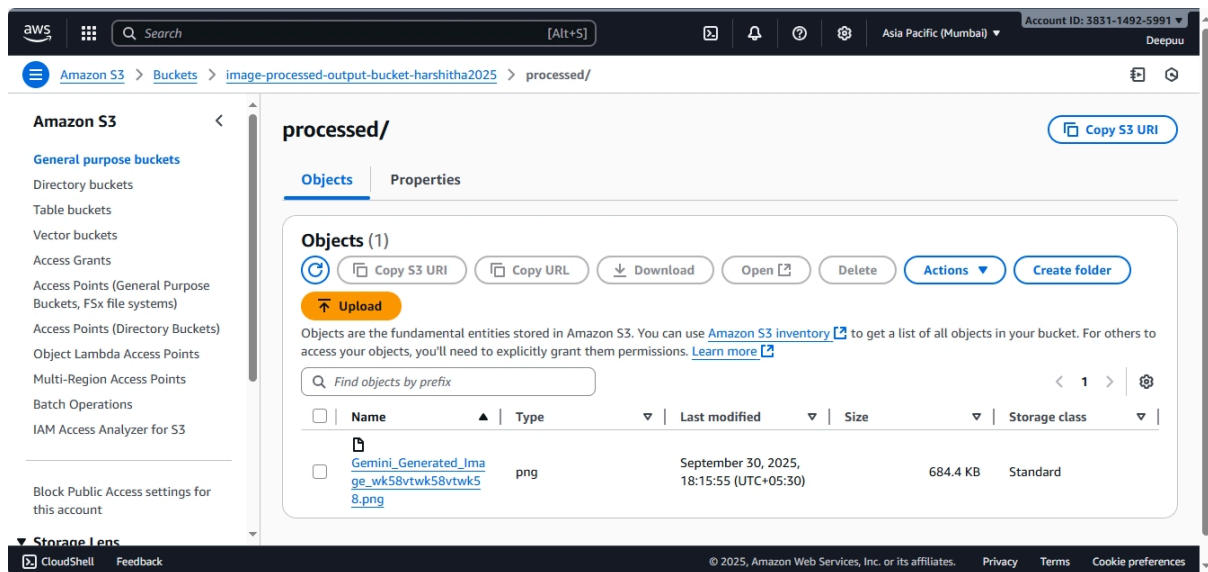
3. Confirm the object appears in uploads / inside your INPUT_BUCKET (S3 console).



4. In Lambda logs (CloudWatch) you should see an invocation and log lines (function prints).



5. Confirm processed image exists in OUTPUT_BUCKET under processed/.



6. Open/download to verify resize/watermark.

If not triggered: check S3 event notification configuration on input bucket and Lambda permission.

10) Troubleshooting & common errors

- **403 AccessDenied when uploading from EB:** Make sure EB environment instances have the instance profile attached and that role has `s3:PutObject` for the input bucket.
- **Lambda fails with ModuleNotFoundError: No module named 'PIL':** Pillow is not present in Lambda runtime — either add Pillow to a Lambda Layer (recommended) or create a deployment package that includes it (build in Amazon Linux environment).
- **Lambda timeout or out of memory:** Increase timeout and memory (Lambda console → Configuration). Large images require more memory and time.
- **Large file uploads (> Lambda / S3 limits):** S3 supports large files via multipart; Lambda / tmp is limited (512 MB default). For huge images or heavy processing, consider using AWS Fargate / ECS or pre-resize on client side.
- **CORS issues on direct browser uploads:** Add a CORS configuration to the bucket.
- **Object key path problems (spaces, plus signs):** Use `urllib.parse.unquote_plus` to decode keys (example code uses this).

11) Cost & cleanup (important)

Costs:

- S3 storage costs (per GB-month) + PUT/GET request costs.
- Lambda: execution time per invocation (small for resize jobs), free tier exists.
- Elastic Beanstalk: runs EC2 instance(s) — the major cost. For testing, choose t3.micro/t3.small or use single-instance environment.

Cleanup commands (quick):

- Delete EB environment & app (console or `eb terminate --all`).
- Delete Lambda function & layer.
- Delete S3 buckets (empty them first) `aws s3 rm s3://${INPUT_BUCKET} --recursive` then `aws s3 rb s3://${INPUT_BUCKET}`
- Remove IAM roles/policies (manual in console or via CLI) — be careful not to remove roles used by other services.

12) Resume lines & interview talking points

Resume bullet (short):

Built an Image Upload & Processing pipeline using Elastic Beanstalk (Flask), S3 and Lambda. Implemented serverless processing (image resize + watermark) using AWS Lambda and S3 event notifications; configured IAM roles, automated deployment with EB CLI and monitored logs with CloudWatch.

Longer explanation for interviews:

- Explain how you separated user-facing interface (EB) from processing (Lambda) to scale compute and reduce cost.
- Describe how permissions were scoped: EB instance profile for upload, Lambda role for read & write to specific buckets, and CloudWatch logging.
- Mention packaging challenges (Pillow native dependencies) and how you solved it with Lambda Layers / container images.